Pontificia Universidad
JAVERIANA
Cali

DIPLOMADO

Arquitectura de Software y Cloud Computing

Educación Continua

Arquitectura del sistema

AWS Serverless y Docker local

Documento de arquitectura y resultados · Reto 2: Alerta de flota vehicular · Módulo 2

Grupo 2
Daniel Mazo
Freddy Aparicio
Camilo Céspedes

Octubre 2026

Contenido

Contenido 2
1. Resumen ejecutivo 3
2. Atributos de calidad y priorización 3
3. Tácticas de arquitectura 4
4. Decisiones de arquitectura 4
5. Contrato HTTP: 200, 503 y 429 5
6. ¿Qué hace el sistema? 5
7. Opciones consideradas 6
8. El plano de control 6
9. Modelo C4 7
9.1 Contexto 7
9.2 Contenedores 7
9.3 Componentes – plano de control 7
9.4 Componentes – AWS 8
9.5 Panorama: AWS y Local lado a lado 8
9.6 Código – equivalencias entre entornos 9
10. Opción AWS – solución oficial 9
10.1 Qué hace y protocolo 9
10.2 Por qué AWS: elasticidad y redundancia por zona 9
10.3 La región se eligió midiendo, no asumiendo 10
10.4 Operación: calentador y cumplimiento de restricciones 10
11. Opción Local – Docker 11
12. Resultados de la medición 11
12.1 Cómo se midió 11
12.2 Resultados de la ejecución oficial (13/10/2026, AWS) 11
13. Disponibilidad – qué pasa si se cae un componente 12
Anexo A. Glosario 13

1. Resumen ejecutivo

El Reto 2 pide un sistema que reciba 1000 eventos de una flota vehicular en 30 segundos, detecte los eventos de tipo Emergency en ese flujo y notifique por correo electrónico a una cuenta Gmail en menos de 15 segundos desde el último envío, respetando un límite de 15 peticiones por segundo en la puerta de entrada y un máximo de 10 instancias de procesamiento activas a la vez.

El foco de este informe es la solución oficial, desplegada de forma serverless en AWS (API Gateway, Lambda, SQS, SES). La misma arquitectura lógica se reprodujo también en local con Docker (nginx, FastAPI, Redis Streams, SMTP), como entorno de desarrollo; ese entorno se documenta en la sección 11, después de la solución oficial.

Todo el documento se organiza alrededor de una sola decisión: qué atributo de calidad se priorizó y a qué costo. El equipo priorizó desempeño (la latencia de la alerta) y escalabilidad (absorber la ráfaga de 1000 eventos), con una restricción que no se negocia: no perder un Emergency. Esa decisión, explicada en la sección 2, es la que explica casi todas las demás: las tácticas de arquitectura (sección 3), las decisiones de diseño (sección 4) y el contrato HTTP del sistema (sección 5) se derivan directamente de ella.

2. Atributos de calidad y priorización

Priorizamos que la alerta llegue rápido (latencia del Emergency al correo) y que el sistema absorba la ráfaga de 1000 eventos escalando procesadores, porque es lo que decide si la alerta sirve: a un conductor en emergencia le importa que alguien se entere rápido.

Atributo	Qué significa aquí	Evidencia
Desempeño (priorizado)	Latencia de la alerta: el correo sale en paralelo a la recepción, la petición nunca espera el envío.	Corrida oficial del 13/10: correo en el buzón en ≤ 1,35 s contra un corte de 15 s.
Escalabilidad (priorizado)	Absorber la ráfaga de 1000 eventos escalando procesadores dentro de un cupo reservado.	Lambda abre instancias según la carga; la cola desacopla el ritmo de entrada del de envío. 1000 peticiones en ~22 s con 6 instancias de ingesta.
Disponibilidad (restricción, no atributo priorizado)	No perder un Emergency. Si no se puede encolar, se responde 503 en vez de un 200 que confirmaría algo que no quedó guardado.	Entre perder y duplicar una alerta, el diseño elige duplicar: un correo repetido molesta, una emergencia perdida no se recupera.

La escalabilidad es el medio para sostener el desempeño bajo la ráfaga; ambos se juzgan con una sola medida, la latencia del correo. La disponibilidad aparece al servicio de la restricción de no perder un Emergency, no como el atributo priorizado en sí, se desarrolla en la sección 13.

Lo que cuesta priorizar así: el 200 OK confirma que la alerta quedó guardada, no que el correo salió ni que llegó; el sistema necesita una pieza más que operar (la cola); y la entrega al menos una vez del correo, sin idempotencia, puede producir un correo duplicado, lo cual es un compromiso aceptado a cambio de no perder alertas.

3. Tácticas de arquitectura

Cada táctica aplicada en un componente concreto de la solución de AWS:

Táctica	Dónde (AWS)	Cómo ayuda	Atributo
Gestionar la llegada de eventos (limitar la tasa)	API Gateway, stage prod: 15 r/s, burst 2000	Absorbe la ráfaga de 1000 sin rechazos y protege la ingesta	Desempeño
Limitar la respuesta al evento (encolar)	SQS, solo para el Emergency	La ingesta responde 200 en milisegundos; el correo sale aparte	Desempeño
Tiempos de ejecución acotados	Timeout de SES 2+3 s; Lambda 15 s	Un correo lento no se come la banda de 15 s	Desempeño
Introducir concurrencia	Instancias de Lambda en paralelo	1000 peticiones en ~22 s con 6 instancias de ingesta	Escalabilidad
Programar recursos (cupo fijo)	Concurrencia reservada por Lambda	Acota cuántas instancias se abren como máximo en el peor escenario; en las ejecuciones oficiales el pico real fue de 6 (sección 10.4)	Escalabilidad
Retry con límite	Notifier: 2 intentos → DLQ	Un fallo pasajero de SES no pierde la alerta	No perder
Detección y manejo de excepciones	Si SQS falla → 503, nunca 200	Quien envía sabe que debe reintentar	No perder
Reintroducción (reclamar pendientes)	Visibilidad de SQS (90 s) y redrive (máx. 3)	Si el notifier muere, el mensaje vuelve a la cola	No perder

4. Decisiones de arquitectura

Decisión	Atributo	Por qué	A costa de
Responder 200 y encolar solo el Emergency	Desempeño	El correo no frena la petición: k6 recibe respuesta en milisegundos	El 200 confirma un mensaje guardado, no enviado; una pieza más (la cola)
Lambda con concurrencia reservada	Escalabilidad	Abre instancias según la carga, dentro de un cupo dimensionado para el peor escenario (10 VUs de k6)	En las ejecuciones oficiales el pico real fue de 8 instancias (6+2), detalle en la sección 10.4

Decisión	Atributo	Por qué	A costa de
SQS + DLQ con reintentos acotados (≤ 12 s)	Restricción: no perder	Si el correo falla, la alerta espera en la cola sin salirse de los 15 s	Se rinde antes: más casos van a la DLQ
Sin idempotencia del mensaje de correo	Restricción: no perder	Entre perder y duplicar, duplicar es lo menos grave.	Correo duplicado posible

5. Contrato HTTP: 200, 503 y 429

Código	Significa	Detalle
200	Válido y guardado	Position: validado y registrado. Emergency: ya encolado. No garantiza que el correo salió ni que llegó.
503	No se pudo encolar	Preferimos un error visible a confirmar una alerta perdida. Quien envía debe reintentar.
429	Se agotó el burst	Lo responde el gateway (nginx o API Gateway) antes de llegar a la ingesta.

6. ¿Qué hace el sistema?

Cada vehículo envía constantemente su posición geográfica y su estado de operación a través de un endpoint. En situaciones críticas, como cuando el conductor presiona un botón de pánico o los sensores detectan una anomalía, se envía un evento de tipo Emergency en lugar de uno de tipo Position. El sistema debe recibir y procesar 1000 eventos correctamente, y enviar una alerta por correo a una cuenta Gmail en el momento en que se reciba un Emergency.

El contrato de entrada es tolerante a propósito: solo el campo type es obligatorio y decide el flujo; si falta la placa del vehículo se registra como DESCONOCIDA; y Emergency se reconoce sin distinguir mayúsculas. El sistema responde en milisegundos y nunca espera al envío del correo para contestar, esa es la decisión de desempeño de la sección 4 aplicada.

Cada evento recibe un identificador (event_id) que une, en los registros, el momento de la recepción, el momento del envío del correo y, para un Emergency, la confirmación de entrega. Ese identificador es el que permite, en la sección 12, seguir una sola alerta de punta a punta entre los distintos componentes.

7. Opciones consideradas

El enunciado no exige una plataforma de cómputo específica, siempre que se respeten los límites de tasa y de instancias simultáneas. Antes de llegar a la solución oficial, el equipo consideró un camino alterno dentro de AWS:

Opción	Qué es
ECS Fargate	Contenedores gestionados por AWS, sin servidores que administrar, pero con cómputo asignado de forma más continua que una función.
AWS serverless con Lambda *	Funciones que solo cobran por invocación y escalan automáticamente dentro de un cupo reservado, sin infraestructura que mantener encendida entre eventos.
Docker local *	La misma arquitectura lógica, con los mismos roles, corriendo en contenedores locales, para iterar sin costo de nube y como respaldo de la demostración en vivo.

** Las dos que se documentan en este informe; ECS Fargate se consideró y se retiró del alcance.*

Entre ECS Fargate y Lambda, Lambda es la opción más adecuada para una carga intermitente como esta, picos cortos de 1000 eventos en 30 segundos y el resto del tiempo sin tráfico, porque se paga por invocación y no por un contenedor que permanece activo. ECS Fargate sería más competitivo si la carga fuera sostenida las 24 horas, donde el costo por invocación de Lambda puede superar al de un contenedor siempre encendido.

8. El plano de control

Un panel web, común a los dos entornos, lanza el script de k6, permite disparar un Emergency manual, muestra la medida de la rúbrica con un semáforo, conserva un historial de ejecuciones y permite descargar los logs (.log / .csv).

Un interruptor (Local / AWS) cambia la fuente de datos sin tocar el resto del panel: en Local lee los logs, Redis y Docker; en AWS lee CloudWatch y el estado de las colas SQS a través de un usuario IAM de solo lectura, y apunta el k6 y el Emergency manual al API Gateway real. En el entorno AWS el panel no tiene permisos para apagar nada: no hay inyección de fallas posible desde ahí.

9. Modelo C4

Dos niveles de enfoque sobre la misma arquitectura con foco en la solución de AWS. El código, en este documento, se limita al mapa de qué archivo equivale a cuál entre los dos entornos, no a explicar el código en sí.

9.1 Contexto



Figura 1. Un solo actor y un solo sistema externo: no hay otras integraciones.

9.2 Contenedores

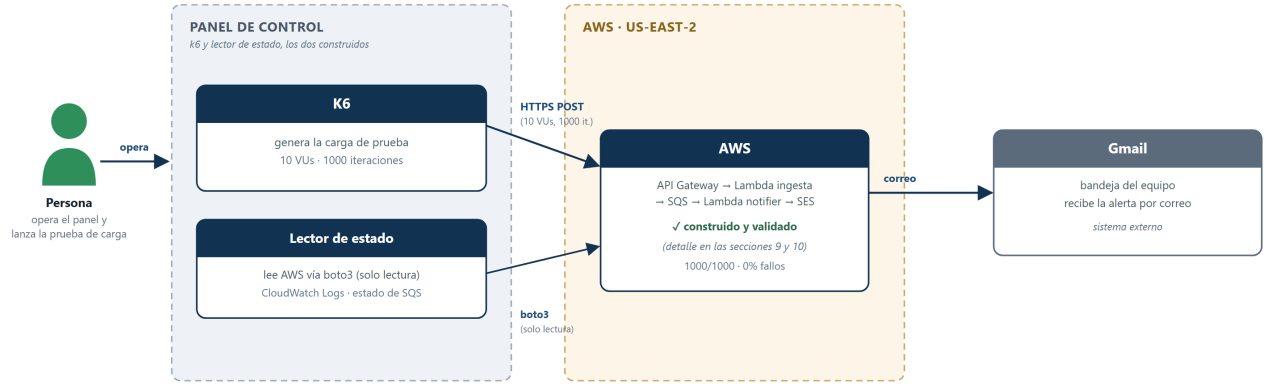


Figura 2. El panel de control (k6 + lector de estado) opera el sistema; la solución de AWS, como contenedor único validado, entrega la alerta a Gmail. El detalle interno de AWS se describe en las figuras 3 y 4.

9.3 Componentes – plano de control

El plano de control tiene un manejador web y componentes con una responsabilidad cada uno: encender/detener el entorno elegido, mostrar el estado de colas y logs, disparar un estímulo manual, lanzar una medición completa ejecutando el k6 real como subproceso, y mantener el historial de ejecuciones. Ninguno participa en lo que se mide: todos viven en el plano de control, no en el camino del evento.

9.4 Componentes – AWS

Al abrir el contenedor de AWS de la figura 2, estos son los componentes reales y su rol en el flujo, con los números de la configuración vigente:



Figura 3. La solución oficial en AWS: API Gateway → Lambda ingesta → SQS → Lambda notifier → SES → Gmail, con CloudWatch Logs, la DLQ y el calentador de EventBridge como soporte. Explicación completa de la concurrencia reservada de 10 frente al pico real en la sección 10.4.

9.5 Panorama: AWS y Local lado a lado

El mismo patrón, columna por columna, en los dos entornos seleccionados; AWS como oficial y Docker Compose en local como referencia, lo único que cambia son los adaptadores de cada rol, no la arquitectura lógica:

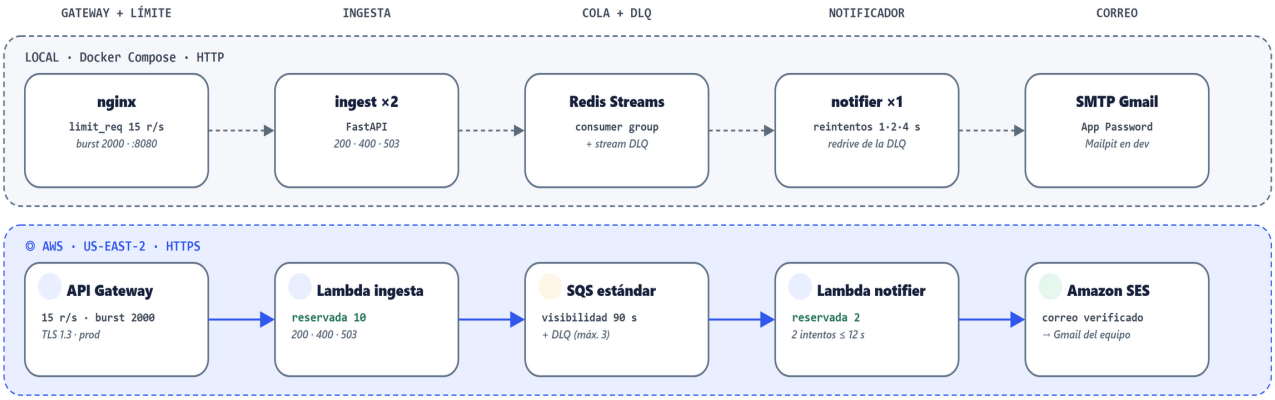


Figura 4. Gateway con límite → ingesta → cola y DLQ → notificador → correo, implementado con nginx/FastAPI/Redis/SMTP en Local y con API Gateway/Lambda/SQS/SES en AWS.

9.6 Código – equivalencias entre entornos

El cuarto nivel de C4, para este documento, muestra qué pieza de un entorno equivale a cuál del otro, no el detalle de cada archivo:

AWS	Local (Docker)
API Gateway (rate 15, burst 2000)	nginx limit_req
Lambda de ingesta	alerta/ingest_app.py (×2 réplicas)
SQS + DLQ	Redis Streams (consumer group, XACK, DLQ)
Lambda notifier	alerta/notifier_worker.py (×1)
SES	SMTP (Mailpit en desarrollo, Gmail en la demo)
CloudWatch Logs	Logs JSON en local

Las dos versiones escriben los mismos mensajes JSON (EVENT_RECEIVED, EMERGENCY_RECEIVED, EMAIL_SENT, EMAIL_RETRY, EMAIL_DLQ…), con hora de Colombia. Esa es la evidencia de las horas de recepción y de envío, en cualquiera de los dos entornos.

10. Opción AWS – solución oficial

10.1 Qué hace y protocolo

Desplegada en la región us-east-2: API Gateway (REST, stage prod, HTTPS, TLS 1.3) aplica el límite de tasa (15 r/s, burst 2000) y reenvía por integración proxy a la función reto2-ingesta. Esta valida el evento y responde de inmediato; si es Emergency, publica un mensaje en la cola estándar SQS reto2-emergencias (visibilidad 90 s). Un disparador, invoca a la función reto2-notifier, que intenta el envío por Amazon SES con un presupuesto acotado: SDK configurado con un solo intento interno y timeouts cortos (conexión de 2 segundos, lectura de 3 segundos), 2 intentos con 1 segundo de espera entre ellos, como máximo 12 segundos antes de desistir.

Si los 2 intentos fallan, la alerta se envía explícitamente a la cola de mensajes fallidos (SQS reto2-emergencias-dlq) en vez de esperar el reintento automático, que llegaría después de la ventana de 15 segundos.

10.2 Por qué AWS: elasticidad y redundancia por zona

Serverless resuelve directamente los dos atributos priorizados en la sección 2: Lambda escala el número de instancias según la carga sin que el equipo aprovisione nada, y se paga solo por lo que se usa, relevante para una carga tan intermitente como 1000 eventos en 30 segundos.

Una región de AWS está compuesta por varias zonas de disponibilidad (data centers físicamente separados). Los servicios gestionados que usa esta solución (API Gateway, Lambda, SQS y SES), replican su infraestructura entre esas zonas de forma automática, sin que el equipo haya configurado nada para

lograrlo. Eso da una redundancia física ante la caída de una sola zona que una instancia única de nginx o de un contenedor local, por diseño, no tiene.

10.3 La región se eligió midiendo, no asumiendo

Todos los recursos (API Gateway, las dos Lambdas, las dos colas SQS, SES, CloudWatch y EventBridge) se desplegaron en la misma región, porque mezclar regiones solo agrega latencia cruzada y complica la identidad de SES, que es por región. Se realizó una prueba contra un endpoint real que existe en toda región (sts.<región>.amazonaws.com), con la misma concurrencia del script k6, 10 VUs, 100 iteraciones por región.

Región	avg (ms)	mediana (ms)	p90 (ms)	p95 (ms)	máx (ms)
us-east-1	222,8	129,0	534,7	543,1	589,8
us-east-2 (elegida)	186,4	111,9	**434,6**	534,6	572,2
us-west-2	215,5	176,3	461,2	537,5	590,3
sa-east-1	201,4	139,5	402,0	465,0	844,7 *

us-east-2 gana en promedio, mediana y p90; sa-east-1 gana por poco en p95, pero tiene el peor máximo de las cuatro (*, un pico aislado, no representativo).

10.4 Operación: calentador y cumplimiento de restricciones

Una regla de EventBridge Scheduler invoca a la función reto2-notifier cada 5 minutos con un evento vacío (el handler lo detecta por la ausencia de Records y no procesa nada), para evitar que el arranque en frío de la función afecte el tiempo de la primera alerta real.

AWS ofrece HTTPS de forma nativa en API Gateway. La concurrencia reservada configurada es de 10 en la función reto2-ingesta. Bajo la carga real de la ejecución oficial se usaron 8 instancias (6 de ingesta más 2 del notifier).

¿Por qué una concurrencia reservada de 10 no se contradice con un pico real de 6? La concurrencia reservada es el cupo dimensionado para el peor escenario posible, no un número que se llena solo porque haya 1000 peticiones en 30 segundos y depende de cuántas estén en vuelo en el mismo instante.

En la ejecución oficial del 13/10, el pico real medido en CloudWatch fue de 6 instancias simultáneas de ingesta. Por eso el valor configurado (10) se mantiene igual al número exacto de VUs del script de k6, es la consideración matemática del peor escenario, aunque una ejecución concreta, por su propio ritmo de llegada, no siempre la alcance.

11. Opción Local – Docker

Entorno secundario: reproduce el mismo patrón que la sección 10 con otros adaptadores: nginx aplica el límite de tasa (limit_req, 15 r/s con burst 2000) y reenvía a dos réplicas de ingesta (FastAPI); cada Emergency se publica en un stream de Redis con un grupo de consumidores; un único proceso notificador lo toma, intenta el envío por SMTP (Mailpit en desarrollo, Gmail en la demo) con reintentos de 1, 2 y 4 segundos, y si los agota, el mensaje pasa a un stream de mensajes fallidos (DLQ). El redrive es automático: cada 15 segundos se reintenta lo que lleva menos de 10 minutos en la DLQ.

En local no se tiene HTTPS: el endpoint de desarrollo es HTTP plano, suficiente para iterar, razón por la cual también se optó por una solución en la nube.

12. Resultados de la medición

12.1 Cómo se midió

Con el script de k6 (10 VUs, 1000 iteraciones, ~30 s) apuntando al endpoint real de cada entorno. El mismo event_id une, en los registros, la recepción del Emergency, el envío del correo (EMAIL_SENT) y, cuando se verificó, la llegada real al buzón mediante la cabecera Received de Gmail. Los logs de ambas Lambdas (o de ambos servicios locales) quedan en JSON, con hora de Colombia en cada línea.

12.2 Resultados de la ejecución oficial (13/10/2026, AWS)

1000/1000 peticiones aceptadas, 0 rechazadas por el límite de tasa (429), 1 evento Emergency, 1 correo enviado al primer intento, 0 mensajes en la DLQ.

Hora (Colombia)	Evento	Fuente
19:01:54.543	EMERGENCY_RECEIVED · placa RJJ-055 (petición 992 de 1000)	CloudWatch · ingesta
19:01:54.654	Última petición de k6 recibida	CloudWatch · ingesta
19:01:54.807	EMERGENCY_ENQUEUED · la ingesta responde 200	CloudWatch · ingesta
19:01:55.298	EMAIL_SENT · SES acepta el correo, intento 1	CloudWatch · notifier
19:01:55	Llega al buzón de Gmail («entregado tras 0 segundos»)	Cabecera Received

13. Disponibilidad – qué pasa si se cae un componente

Si se cae	Mientras está caído	Al volver
Correo (SMTP / SES)	Local: reintentos de 1, 2 y 4 s y luego DLQ. AWS: 2 intentos (≤ 12 s) y luego DLQ.	Local: cada 15 s se reintenta lo que lleva menos de 10 min en la DLQ (EMAIL_REDRIVEN). AWS: queda en la DLQ para revisión manual.
Cola (Redis / SQS)	El Emergency responde 503 y no se guarda. Los Position siguen respondiendo 200.	El notifier se reconecta solo. Lo que recibió 503 no se recupera: debe reenviarlo quien lo originó.
Notifier	Los Emergency se acumulan en la cola, sin perderse.	Local: reclama los pendientes (PENDING_CLAIMED). AWS: SQS los vuelve a ofrecer pasada la visibilidad (90 s).

La disponibilidad aparece aquí al servicio de la restricción, no perder un Emergency, descrita en la sección 2, no como el atributo priorizado. En la versión local se verificó desde el panel: al apagar el SMTP, enviar un Emergency y volver a encenderlo, se observa el redrive desde la DLQ.

Anexo A. Glosario

Término	Definición
Emergency	Tipo de evento que indica una situación crítica (botón de pánico o anomalía detectada). Dispara la notificación por correo.
Position	Tipo de evento de posición y estado normal de un vehículo; no dispara correo.
event_id	Identificador único de cada evento, usado para seguir una misma alerta a través de la recepción, el envío y la entrega del correo.
DLQ (Dead Letter Queue / cola de mensajes fallidos)	Cola donde se guardan los mensajes que no se pudieron procesar o enviar, para revisión, en vez de perderlos en silencio.
Redrive	Reintento automático de un mensaje que volvió a quedar disponible tras agotar su visibilidad o su tiempo en la DLQ.
Visibilidad (SQS)	Tiempo durante el cual un mensaje tomado por un consumidor queda oculto para los demás, mientras se procesa.
Concurrencia reservada	Cupo de instancias que una función Lambda puede usar al mismo tiempo, reservado del total disponible en la cuenta.
Burst / token bucket	Mecanismo de limitación de tasa que permite absorber ráfagas cortas por encima del límite sostenido, hasta agotar un cupo acumulado.
Zona de disponibilidad (AZ)	Centro de datos físicamente independiente dentro de una región de AWS; los servicios gestionados replican entre varias para tolerar la caída de una.
EventBridge Scheduler	Servicio de AWS que invoca una función en un horario programado; aquí se usa para mantener caliente al notificador.
SES (Simple Email Service)	Servicio de envío de correo de AWS, usado por la Lambda notificadora para mandar la alerta.
DMARC	Política de autenticación de correo que verifica si el remitente real coincide con el dominio que dice representar; cuando falla, el correo puede caer en spam.
Idempotencia	Propiedad de una operación que produce el mismo resultado sin importar cuántas veces se repita; este sistema no la implementa para el envío de correo.
k6	Herramienta de pruebas de carga usada para generar los 1000 eventos de la prueba oficial.
VU (usuario virtual)	Unidad de concurrencia de k6: cada VU envía peticiones una tras otra, esperando la respuesta antes de la siguiente.
Táctica de arquitectura	Decisión de diseño puntual, más específica que un patrón, dirigida a mejorar un atributo de calidad concreto (por ejemplo, limitar la tasa de entrada para proteger el desempeño).